DO NOW — RECALL

Lesson 051 · Linked Lists: Nodes and References

Name: _____ Class: _____ Date: _____

1. How does Queue<T> remove items?

2. How does a stack differ from a queue?

3. What is an object reference?

4. A singly linked list is HEAD → node5 → node9 → NULL. A new node7 is available. Describe the two next-pointer assignments needed to insert node7 immediately after node5 without losing access to node9.

DO NOW — CHECK / OBJECTIVES

Correct and improve

Check page 1 in GREEN PEN. Write corrections in full.

1. Model answer

First-in-first-out, from front.

2. Model answer

LIFO versus FIFO.

3. Model answer

Value identifying an object instance.

4. Model answer

Set node7.next = node5.next, then node5.next = node7. Saving the existing successor first preserves access to node9.

By the end, you can…

• A linked-list node holds a value plus one or more links to neighbouring nodes, rather than occupying an indexed fixed-size array slot.

• C# LinkedList<T> is doubly linked: nodes expose Previous and Next; AddAfter inserts relative to an existing node.

• To insert B between A and C, change logical links to A↔B↔C. The operation differs from shifting array elements.

Vocabulary — know these terms

node — Value and link information.

Next — Reference to following node.

Previous — Reference to preceding node.

head — First list node.

NEW LEARNING

Read, annotate and explain

Discover

A linked-list node holds a value plus one or more links to neighbouring nodes, rather than occupying an indexed fixed-size array slot.

Trace

C# LinkedList<T> is doubly linked: nodes expose Previous and Next; AddAfter inserts relative to an existing node.

Evaluate

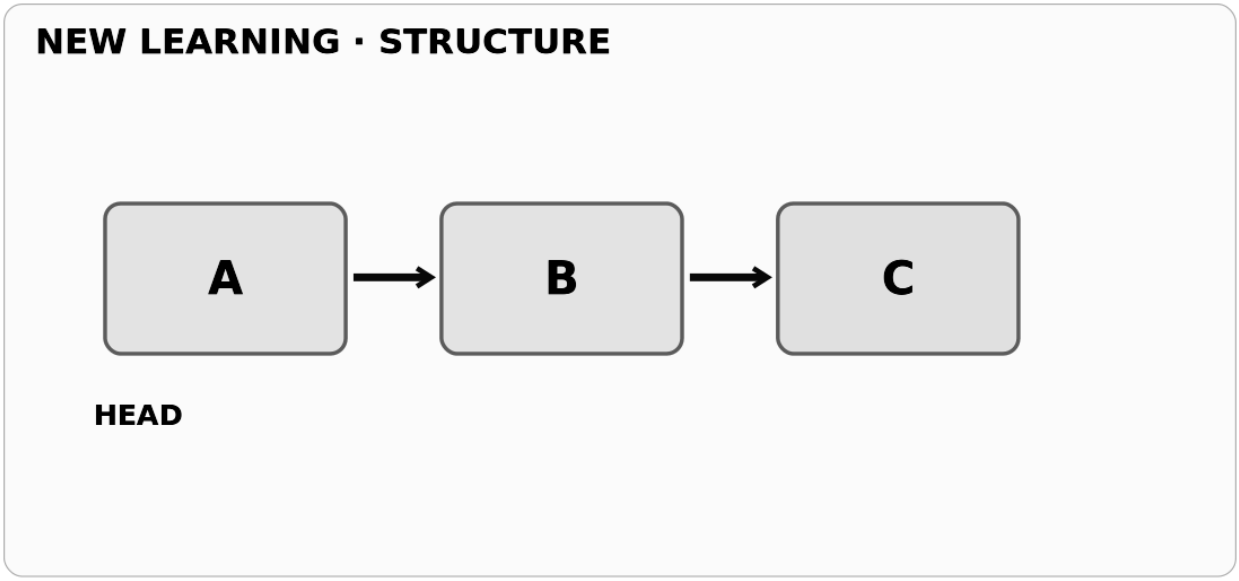
To insert B between A and C, change logical links to A↔B↔C. The operation differs from shifting array elements.

Explain one step in your own words, using the correct technical terms.

New Learning · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

New Learning



NEW LEARNING · STRUCTURE: A, B, C

I DO — WORKED EXAMPLE

Follow the worked method

Trace AddAfter to insert X after A in A→B.

Explain the trace

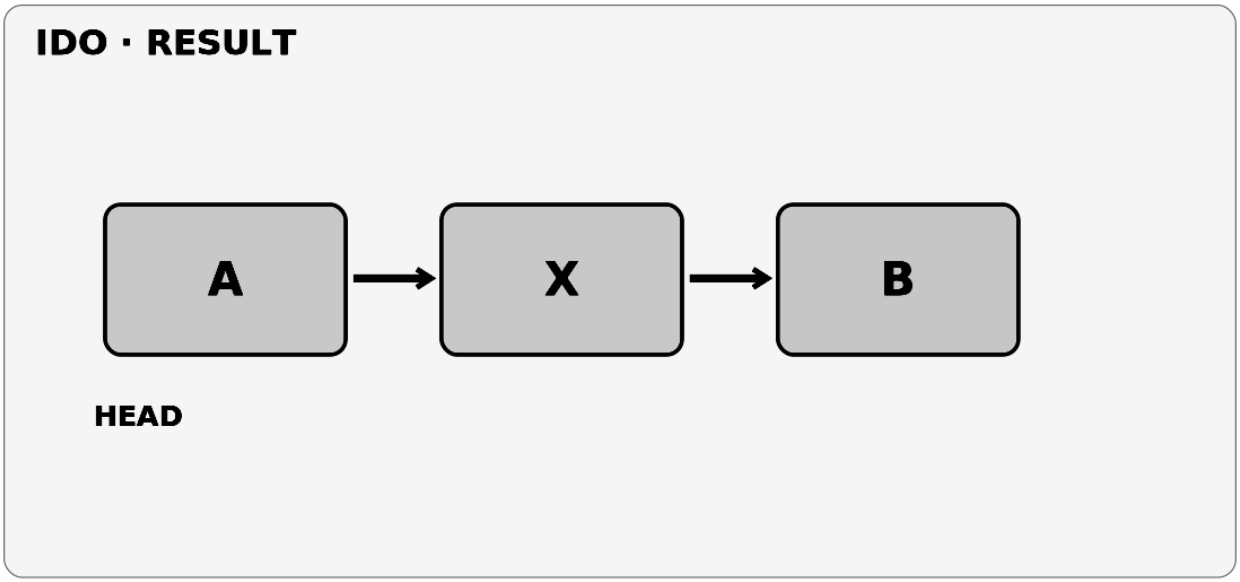
New order A,X,B. The inserted node links between A and B.

I Do · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Trace AddAfter to insert X after A in A→B.

New order A,X,B. The inserted node links between A and B.



IDO · RESULT: A, X, B

WE DO — GUIDED PRACTICE

Try → Discuss → Improve

TRY: Attempt the guided questions in BLACK PEN. DISCUSS: Compare approaches with a partner, explain your reasoning and agree one improvement. IMPROVE: After the Exit Ticket, use the source CHECK model in GREEN PEN to correct and explain errors.

Guided 1 — TRY

Remove the middle node from a different list P→Q→R.
Source code / input:

```
var l=new LinkedList<string>(new[]{"P","Q","R"});
l.Remove("Q");
```

YOU DO — INDEPENDENT PRACTICE

Questions · Part 1

Answer independently. Show code, calculations or a justified explanation.

Question 1

Draw the logical next-links in HEAD→3→6→NULL after inserting 4 after 3. [6]

Question 2

Write C# using LinkedList<int> to create 2,4 then insert 3 between them. [6]

YOU DO — INDEPENDENT PRACTICE

Questions · Part 2

Answer independently. Show code, calculations or a justified explanation.

Question 3

Compare middle insertion in linked list with a C# List<int>, including locating position and access. [6]

EXIT TICKET / REFLECTION

Check your understanding

Complete the actual lesson Exit Ticket before checking.

A linked list always offers O(1) indexed access to its nth node.

Circle: Confident / Getting there / Need help

Explain what you can now do independently.

GREEN PEN — Correct and explain

Choose one error. Write the correct method and explain precisely why your first response was wrong.

WE DO — CHECK AND IMPROVE

Source-grounded model answers

CHECK only after completing the guided practice and Exit Ticket. Use GREEN PEN to improve your method and explain any error.

Guided 1 — CHECK

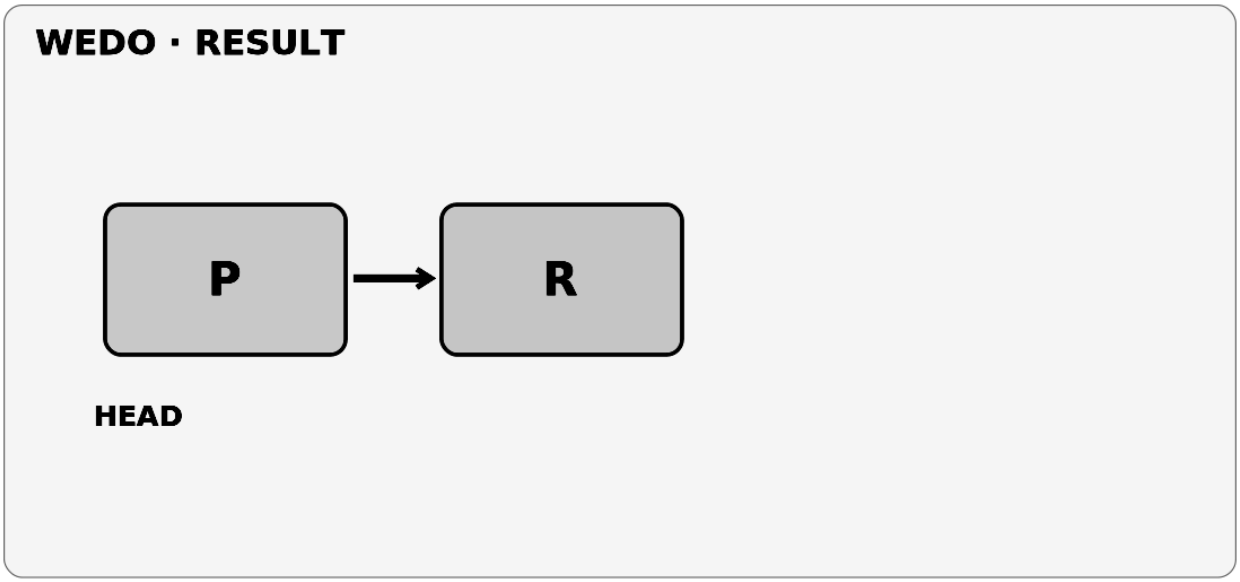
Explain the trace
P,R remain. After deletion P and R are adjacent logical neighbours.

We Do source CHECK · original lesson diagrams

Diagrams appear after the related explanation. Annotate the original source visual.

Remove the middle node from a different list P→Q→R.

P,R remain. After deletion P and R are adjacent logical neighbours.



WEDO · RESULT: P, R